PHP

Unit 1

What You Should Already Know

Before you continue you should have a basic understanding of the following:

- ***HTML***
- ***CSS***
- ***JavaScript***

What is PHP?

- ***PHP is an acronym for "PHP: Hypertext Preprocessor"***
- ***PHP is a widely-used, open source scripting language***
- ***PHP scripts are executed on the server***
- ***PHP is free to download and use***
- ***PHP is a server scripting language, and a powerful tool for making dynamic and interactive Web pages.***
- ***PHP is a widely-used, free, and efficient alternative to competitors such as Microsoft's ASP.***
- ***PHP 7 is the latest stable release.***
- ***PHP code is executed on the server.***

PHP is an amazing and popular language!

It is powerful enough to be at the core of the biggest blogging system on the web (WordPress)!
It is deep enough to run the largest social network (Facebook)!
It is also easy enough to be a beginner's first server side language!

What Can PHP Do?

- ***PHP can generate dynamic page content***
- ***PHP can create, open, read, write, delete, and close files on the server***
- ***PHP can collect form data***
- ***PHP can send and receive cookies***
- ***PHP can add, delete, modify data in your database***
- ***PHP can be used to control user-access***
- ***PHP can encrypt data***

With PHP you are not limited to output HTML. You can output images, PDF files, and even Flash movies. You can also output any text, such as XHTML and XML.

Ashwani mishra

Why PHP?

- *PHP runs on various platforms (Windows, Linux, Unix, Mac OS X, etc.)*
- *PHP is compatible with almost all servers used today (Apache, IIS, etc.)*
- *PHP supports a wide range of databases*
- *PHP is free. Download it from the official PHP resource: www.php.net*
- *PHP is easy to learn and runs efficiently on the server side*

PHP Syntax

A PHP script is executed on the server, and the plain HTML result is sent back to the browser.

Basic PHP Syntax

A PHP script can be placed anywhere in the document. A

PHP script starts with <?php and ends with ?>:

```
<?php
// PHP code goes here
?>
```

The default file extension for PHP files is ".php".

A PHP file normally contains HTML tags, and some PHP scripting code.

Below, we have an example of a simple PHP file, with a PHP script that uses a built-in PHP function "echo" to output the text "Hello World!" on a web page:

Example

```
<!DOCTYPE html>
<html>
<body>

<h1>My first PHP page</h1>

<?php
echo "Hello World!";
?>

</body>
</html>
```

Ashwani mishra

Note: ***PHP statements end with a semicolon (;).***

PHP Case Sensitivity

In PHP, NO keywords (e.g. if, else, while, echo, etc.), classes, functions, and user-defined functions are case-sensitive.

In the example below, all three echo statements below are equal and legal:

Example

```
<!DOCTYPE html>
<html>
<body>

<?php
ECHO "Hello
World!<br>"; echo
"Hello World!<br>";
EcHo "Hello
World!<br>";
?>

</body>
</html>
```

Look at the example below; only the first statement will display the value of the $color variable! This is because $color, $COLOR, and $coLOR are treated as three different variables:

Example

```
<!DOCTYPE html>
<html>
<body>

<?php
$color = "red";
echo "My car is " . $color .
"<br>"; echo "My house is " .
$COLOR . "<br>"; echo "My boat
is " . $coLOR . "<br>";
?>

</body>
</html>
```

Comments in PHP

Ashwani mishra

A comment in PHP code is a line that is not executed as a part of the program. Its only purpose is to be read by someone who is looking at the code.

Ashwani mishra

Comments can be used to:

- ***Let others understand your code***
- ***Remind yourself of what you did - Most programmers have experienced coming back to their own work a year or two later and having to re-figure out what they did. Comments can remind you of what you were thinking when you wrote the code***

PHP supports several ways of commenting:

Example

Syntax for single-line comments:

```
<!DOCTYPE html>
<html>
<body>

<?php
// This is a single-line comment

# This is also a single-line comment
?>

</body>
</html>
```

Example

Syntax for multiple-line comments:

```
<!DOCTYPE html>
<html>
<body>

<?php
/*
This is a multiple-lines comment block that
spans over multiple
lines
*/
?>

</body>
</html>
```

PHP Variables

Creating (Declaring) PHP Variables

In PHP, a variable starts with the $ sign, followed by the name of the variable:

Ashwani mishra

Example

```
<?php
$txt = "Hello world!";
$x = 5;
$y = 10.5;
?>
```

A variable can have a short name (like x and y) or a more descriptive name (age, carname, total_volume).

Rules for PHP variables:

- *A variable starts with the $ sign, followed by the name of the variable*
- *A variable name must start with a letter or the underscore character*
- *A variable name cannot start with a number*
- *A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)*
- *Variable names are case-sensitive ($age and $AGE are two different variables)*

Output Variables

The PHP echo statement is often used to output data to the screen. The

following example will show how to output text and a variable:

Example

```
<?php
$txt =
"W3Schools.com";
echo "I love $txt!";
?>
```

The following example will produce the same output as the example above:

Example

```
<?php
$txt =
"W3Schools.com";
echo "I love " . $txt .
"!";
?>
```

The following example will output the sum of two variables:

Example

```
<?php
$x = 5;
$y = 4;
```

Ashwani mishra

```
echo $x + $y;
?>
```

Ashwani mishra

PHP Variables Scope

In PHP, variables can be declared anywhere in the script.

The scope of a variable is the part of the script where the variable can be referenced/used. PHP

has three different variable scopes:

- *local*
- *global*
- *static*

Global and Local Scope

A variable declared **outside** *a function has a GLOBAL SCOPE and can only be accessed outside a function:*

Example

Variable with global scope:

```
<?php
$x = 5; // global scope

function myTest() {
   // using x inside this function will generate
   an error echo "<p>Variable x inside
   function is: $x</p>";
}
myTest();

echo "<p>Variable x outside function is: $x</p>";
?>
```

A variable declared **within** *a function has a LOCAL SCOPE and can only be accessed within that function:*

Example

Variable with local scope:

```
<?php
function myTest() {
   $x = 5; // local scope
   echo "<p>Variable x inside function is: $x</p>";
}
myTest();
```

Ashwani Mishra

// using x outside the function will generate an error

Ashwani mishra

```
echo "<p>Variable x outside function is: $x</p>";
?>
```

PHP The global Keyword

The global keyword is used to access a global variable from within a function.

To do this, use the global keyword before the variables (inside the function):

Example

```
<?php
$x = 5;
$y = 10;

function myTest() {
  global $x, $y;
  $y = $x + $y;
}

myTest();
echo $y; // outputs 15
?>
```

PHP also stores all global variables in an array called $GLOBALS[index]. The index holds the name of the variable. This array is also accessible from within functions and can be used to update global variables directly.

The example above can be rewritten like this:

Example

```
<?php
$x = 5;
$y = 10;

function myTest() {
  $GLOBALS['y'] = $GLOBALS['x'] + $GLOBALS['y'];
}

myTest();
echo $y; // outputs 15
?>
```

PHP The static Keyword

Normally, when a function is completed/executed, all of its variables are deleted. However,

Ashwani mishra

sometimes we want a local variable NOT to be deleted. We need it for a further job.

Ashwani mishra

To do this, use the static keyword when you first declare the variable:

Example

```
<?php
function
   myTest() {
   static $x = 0;
   echo $x;
   $x++;
}

myTest();
myTest();
myTest();
?>
```

Then, each time the function is called, that variable will still have the information it contained from the last time the function was called.

Strings

A string is series of characters, where a character is the same as a byte. This means that PHP only supports a 256-character set, and hence does not offer native Unicode support.

Syntax

A string literal can be specified in four different ways:

- *single quoted*
- *double quoted*

Single quoted

The simplest way to specify a string is to enclose it in single quotes (the character ').

To specify a literal single quote, escape it with a backslash (\). To specify a literal backslash, double it (\\). All other instances of backslash will be treated as a literal backslash: this means that the other escape sequences you might be used to, such as \r or \n, will be output literally as specified rather than having any special meaning.

```
<?php
echo 'this is a simple string';

?>
```

Double quoted

If the string is enclosed in double-quotes ("), PHP will interpret the following escape sequences for special characters:

```
<?php
echo “this is a simple string”;

?>
```

PHP String Functions

In this chapter we will look at some commonly used functions to manipulate strings.

strlen() - Return the Length of a String

The PHP strlen() function returns the length of a string.

Example

Return the length of the string "Hello world!":

```
<?php
echo strlen("Hello world!"); // outputs 12
?>
```

str_word_count() - Count Words in a String

The PHP str_word_count() function counts the number of words in a string.

Example

Count the number of word in the string "Hello world!":

```
<?php
echo str_word_count("Hello world!"); // outputs 2
?>
```

strrev() - Reverse a String

The PHP strrev() function reverses a string.

Example

Reverse the string "Hello world!":

```
<?php
echo strrev("Hello world!"); // outputs !dlrow olleH
?>
```

strpos() - Search For a Text Within a String

The PHP strpos() function searches for a specific text within a string. If a match is found, the function returns the character position of the first match. If no match is found, it will return FALSE.

Example

Search for the text "world" in the string "Hello world!":

```
<?php
echo strpos("Hello world!", "world"); // outputs 6
?>
```

Tip: The first character position in a string is 0 (not 1).

str_replace() - Replace Text Within a String

The PHP str_replace() function replaces some characters with some other characters in a string.

Example

Replace the text "world" with "Dolly":

```
<?php
echo str_replace("world", "Dolly", "Hello world!"); // outputs Hello Dolly!
?>
```

PHP Data Types

Variables can store data of different types, and different data types can do different things. PHP

supports the following data types:

- *String*
- *Integer*
- *Float (floating point numbers - also called double)*
- *Boolean*
- *Array*
- *Object*
- *NULL*

PHP String

A string is a sequence of characters, like "Hello world!".

A string can be any text inside quotes. You can use single or double quotes:

Ashwani Mishra

Example

```
<?php
$x = "Hello world!";
$y = 'Hello world!';

echo $x;
echo
"<br>";
echo $y;
?>
```

PHP Integer

An integer data type is a non-decimal number between -2,147,483,648 and 2,147,483,647. Rules for integers:

- *An integer must have at least one digit*
- *An integer must not have a decimal point*
- *An integer can be either positive or negative*
- *Integers can be specified in: decimal (base 10), hexadecimal (base 16), octal (base 8), or binary (base 2) notation*

In the following example $x is an integer. The PHP var_dump() function returns the data type and value:

Example

```
<?php
$x = 5985;
var_dump($x);
?>
```

PHP Float

A float (floating point number) is a number with a decimal point or a number in exponential form.

In the following example $x is a float. The PHP var_dump() function returns the data type and value:

Example

```
<?php
$x = 10.365;
var_dump($x);
?>
```

PHP Boolean

A Boolean represents two possible states: TRUE or FALSE.

Ashwani mishra

```
$x = true;
$y = false;
```

Booleans are often used in conditional testing. You will learn more about conditional testing in a later chapter of this tutorial.

PHP Array

An array stores multiple values in one single variable.

In the following example $cars is an array. The PHP var_dump() function returns the data type and value:

Example

```
<?php
$cars =
array("Volvo","BMW","Toyota");
var_dump($cars);
?>
```

PHP Object

An object is a data type which stores data and information on how to process that data. In PHP,

an object must be explicitly declared.

First we must declare a class of object. For this, we use the class keyword. A class is a structure that can contain properties and methods:

Example

```
<?php
class
Car {
  function Car() {
    $this->model = "VW";
  }
}

// create an object
$herbie = new Car();

// show object
properties echo
$herbie->model;
?>
```

PHP NULL Value

Ashwani mishra

Null is a special data type which can have only one value: NULL.

Ashwani mishra

A variable of data type NULL is a variable that has no value assigned to it.

***Tip:** If a variable is created without a value, it is automatically assigned a value of NULL.*

Variables can also be emptied by setting the value to NULL:

Example

```
<?php
$x = "Hello world!";
$x = null;
var_dump($x);
?>
```

PHP Numbers

One thing to notice about PHP is that it provides automatic data type conversion.

So, if you assign an integer value to a variable, the type of that variable will automatically be an integer. Then, if you assign a string to the same variable, the type will change to a string.

This automatic conversion can sometimes break your code.

PHP Integers

An integer is a number without any decimal part.

2, 256, -256, 10358, -179567 are all integers. While 7.56, 10.0, 150.67 are floats.

So, an integer data type is a non-decimal number between -2147483648 and 2147483647. A value greater (or lower) than this, will be stored as float, because it exceeds the limit of an integer.

*Another important thing to know is that even if 4 * 2.5 is 10, the result is stored as float, because one of the operands is a float (2.5).*

Here are some rules for integers:

- *An integer must have at least one digit*
- *An integer must not have a decimal point*
- *An integer can be either positive or negative*
- *Integers can be specified in three formats: decimal (10-based), hexadecimal (16-based - prefixed with 0x) or octal (8-based - prefixed with 0)*

PHP has the following functions to check if the type of a variable is integer:

- *is_int()*
- *is_integer() - alias of is_int()*
- *is_long() - alias of is_int()*

Ashwani mishra

Example

Check if the type of a variable is integer:

```
<?php
$x = 5985;
var_dump(is_int($x));

$x = 59.85;
var_dump(is_int($x));
?>
```

PHP Floats

A float is a number with a decimal point or a number in exponential form. 2.0,

256.4, 10.358, 7.64E+5, 5.56E-5 are all floats.

The float data type can commonly store a value up to 1.7976931348623E+308 (platform dependent), and have a maximum precision of 14 digits.

PHP has the following functions to check if the type of a variable is float:

- *is_float()*
- *is_double() - alias of is_float()*

Example

Check if the type of a variable is float:

```
<?php
$x = 10.365;
var_dump(is_float($x));
?>
```

PHP Infinity

A numeric value that is larger than PHP_FLOAT_MAX is considered infinite. PHP

has the following functions to check if a numeric value is finite or infinite:

- *is_finite()*
- *is_infinite()*

However, the PHP var_dump() function returns the data type and value:

Example

Check if a numeric value is finite or infinite:

Ashwani mishra

```
<?php
$x = 1.9e411;
var_dump($x);
?>
```

PHP Casting Strings and Floats to Integers

Sometimes you need to cast a numerical value into another data type.

The (int), (integer), or intval() function are often used to convert a value to an integer.

Example

Cast float and string to integer:

```
<?php
// Cast float to int
$x = 23465.768;
$int_cast =
(int)$x; echo
$int_cast;

echo "<br>";

// Cast string to int
$x = "23465.768";
$int_cast =
(int)$x; echo
$int_cast;
?>
```

PHP Date and Time

The PHP date() function is used to format a date and/or a time.

The PHP Date() Function

The PHP date() function formats a timestamp to a more readable date and time.

Syntax

```
date(format,timestamp)
```

Parameter	Description
format	Required. Specifies the format of the timestamp

Ashwani mishra

timestamp	*Optional. Specifies a timestamp. Default is the current date and time*

Get a Date

The required format parameter of the date() function specifies how to format the date (or time). Here are some characters that are commonly used for dates:

- *d - Represents the day of the month (01 to 31)*
- *m - Represents a month (01 to 12)*
- *Y - Represents a year (in four digits)*
- *l (lowercase 'L') - Represents the day of the week*

Other characters, like"/", ".", or "-" can also be inserted between the characters to add additional formatting.

The example below formats today's date in three different ways:

Example

```
<?php
echo "Today is " . date("Y/m/d") . "<br>";
echo "Today is " . date("Y.m.d") . "<br>";
echo "Today is " . date("Y-m-d") .
"<br>"; echo "Today is " .
date("l");
?>
```

Get a Time

Here are some characters that are commonly used for times:

- *H - 24-hour format of an hour (00 to 23)*
- *h - 12-hour format of an hour with leading zeros (01 to 12)*
- *i - Minutes with leading zeros (00 to 59)*
- *s - Seconds with leading zeros (00 to 59)*
- *a - Lowercase Ante meridiem and Post meridiem (am or pm) The*

example below outputs the current time in the specified format:

Example

```
<?php
echo "The time is " . date("h:i:sa");
?>
```

Get Your Time Zone

If the time you got back from the code is not correct, it's probably because your server is in another country or set up for a different timezone.

Ashwani Mishra

So, if you need the time to be correct according to a specific location, you can set the timezone you want to use.

The example below sets the timezone to "America/New_York", then outputs the current time in the specified format:

Example

```
<?php
date_default_timezone_set("America/New_Y
ork");    echo "The time is " . date("h:i:sa");
?>
```

Create a Date With mktime()

The optional timestamp parameter in the date() function specifies a timestamp. If omitted, the current date and time will be used (as in the examples above).

The PHP mktime() function returns the Unix timestamp for a date. The Unix timestamp contains the number of seconds between the Unix Epoch (January 1 1970 00:00:00 GMT) and the time specified.

Syntax

mktime(hour, minute, second, month, day, year)

The example below creates a date and time with the date() function from a number of parameters in the mktime() function:

Example

```
<?php
$d=mktime(11, 14, 54, 8, 12, 2014);
echo "Created date is " . date("Y-m-d h:i:sa", $d);
?>
```

PHP Arrays

An array stores multiple values in one single variable:

Example

```
<?php
$cars = array("Volvo", "BMW", "Toyota");
echo "I like " . $cars[0] . ", " . $cars[1] . " and " . $cars[2] . ".";
?>
```

Ashwani mishra

What is an Array?

An array is a special variable, which can hold more than one value at a time.

If you have a list of items (a list of car names, for example), storing the cars in single variables could look like this:

```
$cars1 = "Volvo";
$cars2 = "BMW";
$cars3 = "Toyota";
```

However, what if you want to loop through the cars and find a specific one? And what if you had not 3 cars, but 300?

The solution is to create an array!

An array can hold many values under a single name, and you can access the values by referring to an index number.

Create an Array in PHP

In PHP, the array() function is used to create an array: array();

In PHP, there are three types of arrays:

- **Indexed arrays** - *Arrays with a numeric index*
- **Associative arrays** - *Arrays with named keys*
- **Multidimensional arrays** - *Arrays containing one or more arrays*

Get The Length of an Array - The count() Function

The count() function is used to return the length (the number of elements) of an array:

Example

```
<?php
$cars = array("Volvo", "BMW",
"Toyota"); echo count($cars);
?>
```

Indexed or Numeric Arrays

These type of arrays can be used to store any type of elements, but an index is always a number. By default, the index starts at zero. These arrays can be created in two different ways as shown in the following example:

```
// One way to create an indexed array
```

Ashwani mishra

```
$name_one = array("Zack", "Anthony", "Ram", "Salim", "Raghav");

// Accessing the elements directly
echo "Accessing the 1st array elements
directly:\n"; echo $name_one[2], "\n";
echo $name_one[0],
"\n"; echo
$name_one[4],  "\n";

// Second way to create an indexed array
$name_two[0] = "ZACK";
$name_two[1] = "ANTHONY";
$name_two[2] = "RAM";
$name_two[3] = "SALIM";
$name_two[4] = "RAGHAV";

// Accessing the elements directly
echo "Accessing the 2nd array elements
directly:\n"; echo $name_two[2], "\n";
echo $name_two[0],
"\n"; echo
$name_two[4],  "\n";

?>
```

Output:

```
Accessing the 1st array elements directly: Ram
Zack
Raghav
Accessing the 2nd array elements directly: RAM
ZACK
RAGHAV
```

Traversing: *We can traverse an indexed array using loops in PHP. We can loop through the indexed array in two ways. First by using for loop and secondly by using foreach. You can refer to* ***PHP | Loops*** *for the syntax and basic use.*

Example:

```
<?php

// Creating an indexed array
$name_one = array("Zack", "Anthony", "Ram", "Salim", "Raghav");

// One way of Looping through an array
usign foreach echo "Looping using foreach:
\n";
foreach ($name_one as
```

```
    $val){ echo $val. "\n";
}
```

Ashwani mishra

```
// count() function is used to count
// the number of elements in an array
$round = count($name_one);
echo "\nThe number of elements are $round \n";

// Another way to loop through the array
using for echo "Looping using for: \n";
for($n = 0; $n < $round;
   $n++){ echo
   $name_one[$n], "\n";
}

?>
```

Output:

```
Looping using foreach: Zack
Anthony
Ram
Salim
Raghav

The number of elements are 5 Looping
using for:
ZACK
ANTHONY
RAM
SALIM
RAGHAV
```

Associative Arrays

These type of arrays are similar to the indexed arrays but instead of linear storage, every value can be assigned with a user-defined key of string type.

Example:

```
<?php

// One way to create an associative array
$name_one = array("Zack"=>"Zara",
            "Anthony"=>"Any", "Ram"=>"Rani",
            "Salim"=>"Sara", "Raghav"=>"Ravina");

// Second way to create an associative array
```

```
$name_two["zack"] = "zara";
$name_two["anthony"] = "any";
```

Ashwani mishra

```
$name_two["ram"] = "rani";
$name_two["salim"] = "sara";
$name_two["raghav"] = "ravina";

// Accessing the elements directly
echo "Accessing the elements
directly:\n"; echo
$name_two["zack"], "\n";
echo $name_two["salim"],
"\n"; echo
$name_two["anthony"], "\n";
echo $name_one["Ram"],
"\n"; echo
$name_one["Raghav"], "\n";

?>
```

Output:

```
Accessing the elements directly: zara

sara

any

Rani

Ravina
```

Traversing Associative Arrays: *We can traverse associative arrays in a similar way did in numeric arrays using loops. We can loop through the associative array in two ways. First by using for loop and secondly by using foreach. You can refer to PHP | Loops for the syntax and basic use.*

Example:

```
<?php

// Creating an associative array
$name_one = array("Zack"=>"Zara",
                "Anthony"=>"Any", "Ram"=>"Rani",
                "Salim"=>"Sara", "Raghav"=>"Ravina");

// Looping through an array using
foreach echo "Looping using
foreach: \n";
foreach ($name_one as $val => $val_value){
   echo "Husband is ".$val." and Wife is ".$val_value."\n";
}

// Looping through an array
using for echo "\nLooping
using for: \n";
$keys = array_keys($name_two);
$round = count($name_two);

for($i=0; $i < $round; ++$i) {
```

```
    echo $keys[$i] . ' ' . $name_two[$keys[$i]] . "\n";
}

?>
```

Output:

Ashwani mishra

```
Looping using foreach:
Husband is Zack and Wife is Zara Husband
is Anthony and Wife is Any Husband is Ram
and Wife is Rani Husband is Salim and Wife
is Sara Husband is Raghav and Wife is
Ravina

Looping using for:
zack zara  anthony
any
ram rani
salim sara
raghav ravina
```

Multidimensional Arrays

Multi-dimensional arrays are such arrays which stores an another array at each index instead of single element. In other words, we can define multi-dimensional arrays as array of arrays. As the name suggests, every element in this array can be an array and they can also hold other sub-arrays within. Arrays or sub-arrays in multidimensional arrays can be accessed using multiple dimensions. Example:

```
<?php

// Defining a multidimensional array
$favorites =
   array( array(
      "name" => "Dave Punk",
      "mob" => "5689741523",
      "email" => "davepunk@gmail.com",
   ),
   array(
      "name" => "Monty
      Smith", "mob" =>
      "2584369721",
      "email" => "montysmith@gmail.com",
   ),
   array(
      "name" => "John
      Flinch", "mob" =>
      "9875147536",
      "email" => "johnflinch@gmail.com",
   )
);

// Accessing elements
echo "Dave Punk email-id is: " .
```

```
$favorites[0]["email"], "\n"; echo "John Flinch
mobile number is: " . $favorites[2]["mob"];
```

Ashwani mishra

```
?>
```

Traversing Multidimensional Arrays: *We can traverse through the multidimensional array using for and foreach loop in a nested way. That is, one for loop for the outer array and one for loop for the inner array.*

Example:

```
<?php
// Defining a multidimensional array
$favorites = array(
   "Dave Punk" =>
   array(
      "mob" => "5689741523",
      "email" => "davepunk@gmail.com",
   ),
   "Dave Punk" => array(
      "mob" =>
      "2584369721",
      "email" => "montysmith@gmail.com",
   ),
   "John Flinch" => array(
      "mob" =>
      "9875147536",
      "email" => "johnflinch@gmail.com",
   )
);

// Using for and foreach in nested form
$keys = array_keys($favorites);
for($i = 0; $i < count($favorites);
   $i++) { echo $keys[$i] . "\n";
   foreach($favorites[$keys[$i]] as $key =>
      $value) {  echo $key . " : " . $value . "\n";
   }
   echo  "\n";
}

?>
```

Output:

```
Dave Punk
mob : 2584369721
email : montysmith@gmail.com

John Flinch
mob : 9875147536
email : johnflinch@gmail.com
```

array_count_values() Function

Ashwani mishra

the array_count_values() function counts all the values of an array.

Ashwani mishra

Example

Count all the values of an array:

```
<?php
$a=array("A","Cat","Dog","A","Dog");
print_r(array_count_values($a));
?>
```

array_reverse() Function

The array_reverse() function returns an array in the reverse order.

```
<?php

$a=array(1,2,3,4);

print_r(array_reverse(

$a));

?>
```

array_search() Function

The array_search() function search an array for a value and returns the key. *Example*

Search an array for the value "red" and return its key:

```
<?php
$a=array("a"=>"red","b"=>"green","c"=
>"blue");  echo
array_search("red",$a);
?>
```

```
<?php

$a=array(1,2,3,4,5);

echo array_search("2",$a);

?>
```

array_sum() Function

Ashwani mishra

The array_sum() function returns the sum of all the values in the array.

Ashwani mishra

Example

Return the sum of all the values in the array (5+15+25):

```
<?php
$a=array(5,15,25);
echo
array_sum($a);
?>
```

sort() Function

The sort() function sorts an indexed array in ascending order.

Sort the elements of the $cars array in ascending alphabetical order:

```
<?php
$cars=array("Volvo","BMW","Toy
ota"); sort($cars);
?>
```

asort() Function

The asort() function sorts an associative array in ascending order, according to the value. *Sort an associative array in ascending order, according to the value:*

```
<?php
$age=array("Peter"=>"35","Ben"=>"37","Joe"=>"
43"); asort($age);
?>
```

arsort() Function

The arsort() function sorts an associative array in descending order, according to the value. *Sort an associative array in descending order, according to the value:*

```
<?php
$age=array("Peter"=>"35","Ben"=>"37","Joe"=>"
43"); arsort($age);
?>
```

krsort() Function

Sort an associative array in descending order, according to the key:

Ashwani mishra

```
<?php
$age=array("Peter"=>"35","Ben"=>"37","Joe"=>"
43");   krsort($age);
```

Ashwani mishra

```
?>
```

The krsort() function sorts an associative array in descending order, according to the key.

count() Function

The count() function returns the number of elements in an array. *Return the number of elements in an array:*

```
<?php
$cars=array("Volvo","BMW","Toyot
a");   echo count($cars);
?>
```

PHP Conditional Statements

Very often when you write code, you want to perform different actions for different conditions. You can use conditional statements in your code to do this.

In PHP we have the following conditional statements:

- *if statement - executes some code if one condition is true*
- *if...else statement - executes some code if a condition is true and another code if that condition is false*
- *if...elseif...else statement - executes different codes for more than two conditions*
- *switch statement - selects one of many blocks of code to be executed*

PHP - The if Statement

The if statement executes some code if one condition is true.

Syntax

```
if (condition) {
   code to be executed if condition is true;
}
```

Example

Output "Have a good day!" if the current time (HOUR) is less than 20:

```
<?php
$t = date("H");

if ($t < "20") {
   echo "Have a good day!";
}
?>
```

Ashwani mishra

PHP - The if...else Statement

The if...else statement executes some code if a condition is true and another code if that condition is false.

Syntax

```
if (condition) {
   code to be executed if condition is true;
} else {
   code to be executed if condition is false;
}
```

Example

Output "Have a good day!" if the current time is less than 20, and "Have a good night!" otherwise:

```
<?php
$t = date("H");

if ($t < "20") {
   echo "Have a good day!";
} else {
   echo "Have a good night!";
}
?>
```

PHP - The if...elseif...else Statement

The if...elseif...else statement executes different codes for more than two conditions.

Syntax

```
if (condition) {
   code to be executed if this condition is true;
} elseif (condition) {
   code to be executed if first condition is false and this condition is true;
} else {
   code to be executed if all conditions are false;
}
```

Example

Output "Have a good morning!" if the current time is less than 10, and "Have a good day!" if the current time is less than 20. Otherwise it will output "Have a good night!":

```
<?php
$t = date("H");
```

Ashwani mishra

```
if ($t < "10") {
   echo "Have a good morning!";
} elseif ($t < "20") {
   echo "Have a good day!";
} else {
   echo "Have a good night!";
}
?>
```

The PHP switch Statement

Use the switch statement to select one of many blocks of code to be executed.

Syntax

```
switch (n)
{
   case label1:
      code to be executed if n=label1;
      break;
   case
   label2:
      code to be executed if n=label2;
      break;
   case
   label3:
      code to be executed if n=label3;
      break;
   ...
   default:
      code to be executed if n is different from all labels;
}
```

This is how it works: First we have a single expression n (most often a variable), that is evaluated once. The value of the expression is then compared with the values for each case in the structure. If there is a match, the block of code associated with that case is executed. Use break to prevent the code from running into the next case automatically. The default statement is used if no match is found.

Example

```
<?php
$favcolor = "red";

switch
   ($favcolor) {
   case "red":
```

Ashwani mishra

```
    echo "Your favorite color is
    red!"; break;
case "blue":
    echo "Your favorite color is blue!";
```

Ashwani mishra

```
    break;
  case
  "green":
    echo "Your favorite color is
    green!"; break;
  default:
    echo "Your favorite color is neither red, blue, nor green!";
}
?>
```

PHP Loops

Often when you write code, you want the same block of code to run over and over again a certain number of times. So, instead of adding several almost equal code-lines in a script, we can use loops.

Loops are used to execute the same block of code again and again, as long as a certain condition is true.

In PHP, we have the following loop types:

- *while - loops through a block of code as long as the specified condition is true*
- *do...while - loops through a block of code once, and then repeats the loop as long as the specified condition is true*
- *for - loops through a block of code a specified number of times*
- *foreach - loops through a block of code for each element in an array*

The PHP while Loop

The while loop executes a block of code as long as the specified condition is true.

Syntax

```
while (condition is true) {
  code to be executed;
}
```

Examples

The example below displays the numbers from 1 to 5:

Example

```
<?php
$x = 1;

while($x <= 5) {
  echo "The number is: $x <br>";
  $x++;
}
```

Ashwani mishra

```
?>
```

PHP do while Loop

The do...while loop - Loops through a block of code once, and then repeats the loop as long as the specified condition is true.

The do...while loop will always execute the block of code once, it will then check the condition, and repeat the loop while the specified condition is true.

Synta

x do

```
{
   code to be executed;
} while (condition is
```

true); Examples

The example below first sets a variable $x to 1 ($x = 1). Then, the do while loop will write some output, and then increment the variable $x with 1. Then the condition is checked (is $x less than, or equal to 5?), and the loop will continue to run as long as $x is less than, or equal to 5:

Example

```
<?php
$x = 1;

do {
   echo "The number is: $x <br>";
   $x++;
} while ($x <= 5);
?>
```

for Loop

The for loop - Loops through a block of code a specified number of times.

The for loop is used when you know in advance how many times the script should run.

Syntax

```
for (init counter; test counter; increment counter) {
   code to be executed for each iteration;
}
```

Parameters:

- *init counter: Initialize the loop counter value*
- *test counter: Evaluated for each loop iteration. If it evaluates to TRUE, the loop continues. If it*

Ashwani Mishra

evaluates to FALSE, the loop ends.

- ***increment counter: Increases the loop counter value***

Ashwani mishra

Examples

The example below displays the numbers from 0 to 10:

Example

```
<?php
for ($x = 0; $x <= 10; $x++)
   { echo "The number is:
   $x <br>";
}
?>
```

PHP foreach Loop

The foreach loop works only on arrays, and is used to loop through each key/value pair in an array.

Syntax

```
foreach ($array as $value) {
  code to be executed;
}
```

For every loop iteration, the value of the current array element is assigned to $value and the array pointer is moved by one, until it reaches the last array element.

Examples

The following example will output the values of the given array ($colors):

Example

```
<?php
$colors = array("red", "green", "blue", "yellow");

foreach ($colors as $value)
  { echo "$value <br>";
}
?>
```

Ashwani mishra